

Compressing Search Spaces for Fast Subgraph Matching via Vertex-Set Simulation

Zhenqi Su
Beihang University
Beijing, China
szq02@buaa.edu.cn

Ping Lu
Beihang University
Beijing, China
luping@buaa.edu.cn

Ting Deng
Beihang University
Beijing, China
dengting@buaa.edu.cn

Yutong Ye
Beihang University
Beijing, China
yutongye@buaa.edu.cn

Chao Tian
The Queen's University Belfast
Northern Ireland, United Kingdom
c.tian@qub.ac.uk

Richong Zhang
Beihang University
Beijing, China
zhangrichong@buaa.edu.cn

ABSTRACT

Subgraph matching is a fundamental operation in graph analytics, but its NP-hardness makes exhaustive enumeration expensive, especially on large and highly symmetric graphs. Most backtracking-based algorithms assign query vertices to vertices in a graph one at a time, which often leads to redundant exploration when many candidates are locally indistinguishable. To reduce such redundancy, we propose VSMatch, a subgraph matching algorithm that compresses the search space before enumeration. VSMatch is built on Vertex-set Simulation (VSSim), a structural filtering model that groups candidate vertices with the same neighbors with respect to the query while preserving matching results. Then VSMatch performs enumeration by mapping each query vertex to a set of vertices, which reduces the searching space. Extensive experiments on real-world and synthetic datasets show that VSMatch outperforms state-of-the-art subgraph matching algorithms, achieving improvements of up to 23 orders of magnitude in embeddings-per-second (EPS) throughput.

PVLDB Reference Format:

Zhenqi Su, Ping Lu, Ting Deng, Yutong Ye, Chao Tian, and Richong Zhang. Compressing Search Spaces for Fast Subgraph Matching via Vertex-Set Simulation. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The full version, source code, data, and/or other artifacts have been made available at <https://github.com/SU-ZQ/VSMatch>.

1 INTRODUCTION

Graphs are widely used to model complex relationships among entities in real-world systems. As a fundamental operation in graph analytics, subgraph matching has been extensively studied and applied in social network analysis [3, 16, 52], biochemical data processing [9, 15], knowledge graphs [4, 36], fraud detection [20, 48], and

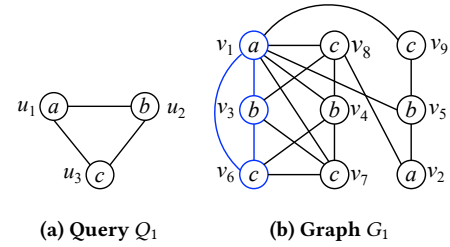


Figure 1: A running example of subgraph matching

graph databases [37]. Given a query and a graph, subgraph matching aims to enumerate all bijective mappings from the query to subgraphs of the graph under isomorphic semantic. For the query Q_1 and the graph G_1 in Figure 1, one match from Q_1 to the subgraph of G_1 highlighted in blue is defined as $u_1 \mapsto v_1, u_2 \mapsto v_3$ and $u_3 \mapsto v_6$.

Despite its broad applicability, subgraph matching is NP-hard [22], and the number of valid matches may grow exponentially with the query size. To improve the performance, modern subgraph matching algorithms typically follow a filtering-ordering-enumeration paradigm [38]. Representative algorithms, such as TurboISO [25], VF3 [11], DPiso [24], CECI [7] and BICE [12], have developed effective technologies for candidate filtering, matching ordering, and search-space pruning. Nevertheless, most existing methods still rely on a vertex-level backtracking procedure, where each recursive step assigns one query vertex to one candidate vertex in the graph. Such vertex-by-vertex enumeration may generate redundant branches when multiple candidate vertices are locally indistinguishable. Figure 2 illustrates the searching tree of the query Q_1 in graph G_1 from Figure 1. Observe that vertices v_3 and v_4 have identical neighborhoods, but vertex-by-vertex enumeration explores them separately, causing redundant checks during recursive matching. Such redundancy becomes particularly severe on large and highly symmetric graphs, leading to deep recursive search trees and repeated checking.

Several recent methods have attempted to exploit structural redundancy during enumeration. Algorithm VEQ [35] uses dynamic equivalences to support backward-looking pruning, but it still performs vertex-by-vertex backtracking and requires additional conflict-recording structures. Algorithm B $\odot$ X [40] groups structurally similar data vertices into search boxes and processes them in batches, but its enumeration is coupled with a recursive depth-first

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

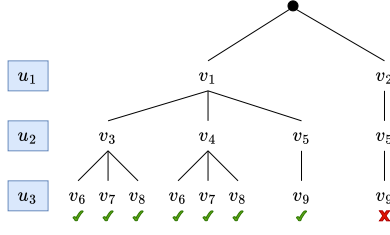


Figure 2: The search tree of traditional backtracking algorithms for query Q_1 and graph G_1 in Figure 1.

procedure, and the boxes are repeatedly extracted, refined, and backtracked at different search depths. Can we decouple the similarity grouping from the recursive procedure to avoid repeated checking?

In this paper, we propose VSMATCH, a novel subgraph matching algorithm that compresses the candidate space before enumeration. At its core is the notation *Vertex-Set Simulation* (VSSim), which groups locally indistinguishable candidates while preserving matching results. After grouping the vertices, we construct a contraction graph to compactly represent the original graphs, which accelerates the enumeration. Given the contraction graphs, we can exploit any existing algorithms to enumerate the matches. In this paper, we take IVE [31] as a case study, to show how to incorporate existing algorithms with the contraction graphs.

Contributions. Our contributions are summarized as follows:

- (1) We present a new graph simulation model, referred to as *Vertex-Set Simulation*. Given a query Q and a graph G , it computes a binary relation $\mathcal{R} \subseteq V_Q \times 2^{V_G}$, such that for each $(u, V) \in \mathcal{R}$, vertices in V have the same label as the query vertex u , and are locally indistinguishable with respect to the edges incident to u in Q , allowing them to be treated as a vertex during subgraph matching. We show that there exists a unique minimum maximal vertex-set simulation relation, and develop an algorithm to compute unique relation.
- (2) We propose a contraction graph G_c from a graph G and a query Q based on the minimum maximal vertex-set simulation relation $\mathcal{R}$. Intuitively, for each pair $(u, V) \in \mathcal{R}$, we create a vertex v_V in G_c to represent the vertex set V . The contraction graph G_c is much smaller than the original graph G , which accelerates the enumeration. Moreover, we show that G_c is sound for the enumeration, i.e., all matches of Q in G can be enumerated using G_c and the vertex sets represented by its vertices, without accessing G .
- (3) We develop VSMATCH, a subgraph matching algorithm that enumerates all matches of Q in G using the contraction graph G_c . VSMATCH first performs enumeration on the compressed search space induced by G_c , and then expands each compressed match into matches of Q in G . Actually, any existing subgraph matching algorithms can be adapted with minimum changes to compute the matches of Q in G using G_c . In this paper, we develop VSMATCH by extending Algorithm IVE [31], as a case study.
- (4) We conduct extensive experiments on real-world and synthetic datasets, and found the following. (a) VSMATCH consistently outperforms state-of-the-art algorithms and achieves up to 23 orders of magnitude improvement in EPS metric. (b) Vertex-set simulation is effective. Given a graph G with 3.8M vertices and 16.5M edges, it

Table 1: Summary of notations.

Notation	Description
$G/G_c/Q$	graph /contraction graph/query
(V_G, E_G, L_G)	undirected labeled graph
$N_G(v) / N_Q(u)$	The set of neighbors of v in G / u in Q
d_G/d_Q	maximum degrees of G/Q
$Q \preceq_{vs} G$	G matches Q via <i>vertex-set simulation</i>
$\mathcal{R}$	a binary <i>relation</i> for vertex-set simulation
$C(u)$	all vertices having the same label as u
$C_{\mathcal{R}}(u)$	vertices matched by u in vertex-set simulation
M/M_p	<i>full match</i> /partial match of Q in G

can effectively filter invalid candidates, and on average results in contraction graphs with 4194 vertices and 5241 edges. (c) Vertex-set simulation can be computed efficiently. It takes only 0.3s to process a graph with 3M vertices and 16M edges, and accounts for 4.89% of overall time of VSMATCH. (d) VSMATCH scales well. It takes only 196.91s to enumerate all matches of a query with 10 vertices on a graph with 2M vertices and 42M edges.

2 PRELIMINARIES

Let Γ be a countably infinite set of labels.

Graphs. We consider *undirected labeled* $G = (V_G, E_G, L_G)$, where (1) V_G is a finite set of vertices; (2) $E_G \subseteq V_G \times V_G$ is a finite set of edges, where (v, v') is an edge between vertices v and v' ; and (3) L_G is a labeling function that assigns a label $L_G(v) \in \Gamma$ to each vertex v in V_G .

Let $N_G(v)$ denote the set of neighbors of vertex $v \in V_G$, i.e., $N_G(v) = \{v' \mid (v, v') \in E_G\}$. A graph $G' = (V_{G'}, E_{G'}, L_{G'})$ is a *subgraph* of G if $V_{G'} \subseteq V_G$, $E_{G'} \subseteq E_G$, and for each $v \in V_{G'}$, $L_{G'}(v) = L_G(v)$.

Queries. A query is also an undirected labeled graph $Q = (V_Q, E_Q, L_Q)$, where (a) V_Q (resp. E_Q) is a finite set of query vertices (resp. query edges), and (b) L_Q assigns a label $L_Q(u) \in \Gamma$ to each query vertex $u \in V_Q$. As commonly assumed in subgraph matching, we assume that all queries are connected.

Subgraph matching. We next define subgraph isomorphism. Consider a graph $G = (V_G, E_G, L_G)$ and a query $Q = (V_Q, E_Q, L_Q)$.

Subgraph matches. A *full match* M of query Q in the graph G is a bijective mapping from V_Q to V_G of a subgraph $G' = (V_{G'}, E_{G'}, L_{G'})$ of G such that (1) for each $u \in V_Q$, u and $M(u)$ carry the same label, i.e., $L_Q(u) = L_G(M(u))$; and (2) the adjacency relation preserves, i.e., $(u, u') \in E_Q$ if and only if $(M(u), M(u')) \in E_{G'}$.

A *partial match* of Q in G is an injective mapping M_p from a *subgraph* of Q to a subgraph of G as above. The partial match M_p is called *valid* if it can be extended to be a full match M of Q in G such that for each $u \in V_Q$, $M(u) = M_p(u)$ if $M_p(u)$ is defined.

Denote by $Q(G)$ the set of all full matches of Q in G .

Problem definition. The paper studies the subgraph matching problem, which is stated as follows.

- **Input:** A query Q and a graph G .
- **Output:** The set $Q(G)$ of all full matches of Q in G .

Graph simulation. Graph simulation and its extensions can be used as a lightweight filter for subgraph matching (e.g., [19, 24, 35]).

Given a query Q and a graph G , *graph simulation* is to compute a binary relation $R \subseteq V_Q \times V_G$ such that (1) for each query vertex $u \in V_Q$, there exists a vertex $v \in V_G$ such that $(u, v) \in R$; and (2) for each pair $(u, v) \in R$, (a) u and v have the same label, i.e., $L_Q(u) = L_G(v)$, and (b) for each query edge (u, u') in E_Q , there exists an edge (v, v') in graph G such that $(u', v') \in R$. It is known that there exists a unique maximum relation for Q and G , which can be computed in quadratic time [28].

Graph simulation provides a sound filter for subgraph matching. Indeed, if a data vertex v matches a query vertex u via subgraph isomorphism, then (u, v) satisfies the simulation constraints, and thus must be contained in the maximum simulation relation. Since this relation can be computed in polynomial time, it can efficiently prune invalid candidates before backtracking. In practice, a few refinement iterations are sufficient to remove most invalid candidates [24].

Example 1: For the query Q_1 and graph G_1 in Figure 1. One full match M is defined as follows: $u_1 \mapsto v_1$, $u_2 \mapsto v_3$ and $u_3 \mapsto v_6$. The maximum simulation relation for Q and G is $R = \{(u_1, v_1), (u_1, v_2), (u_2, v_3), (u_2, v_4), (u_2, v_5), (u_3, v_6), (u_3, v_7), (u_3, v_8), (u_3, v_9)\}$. All pairs (u_1, v_1) , (u_2, v_3) and (u_3, v_6) from M are contained in R . $\square$

In this paper, we extend graph simulation from vertex-level relations to vertex-set relations (see Section 3), which associated each query vertex with a vertex subset of the graph. Vertices in the subset are locally indistinguishable with respect to the edges incident to u in Q , allowing them to be treated as an unit during subgraph matching, which accelerates the enumeration of full matches.

3 VERTEX-SET SIMULATION

We define vertex-set simulation to group vertices in the graph G to accelerate the enumeration in subgraph matching.

Challenges. We can use graph simulation and its extensions to filter candidates before the subgraph enumeration, as in [19, 24, 35]. However, this strategy faces the following two limitations. (1) When checking whether one vertex v can match a query vertex u , graph simulation only inspects the edges incident to v and u , and ignores the edges between their neighbors (i.e., triangle relation [42]). For example, a query vertex that participates in a triangle may be mapped to a vertex that is not contained in any triangle in the graph (e.g., the pair (u_1, v_2) in Example 1). Can we filter such candidates by extending graph simulation? (2) Graph simulation checks whether each query vertex v can match a vertex u individually. However, multiple vertices $v_1, v_2, \dots, v_n$ with the same label may have the same neighbors w.r.t. the candidates of u 's neighbors, then they are indistinguishable under graph simulation. Can we group these vertices and treat them as a vertex to avoid redundant computation?

We adopt the following strategies to address the challenges.

- (1) We define a relation $\mathcal{R} \subseteq V_Q \times 2^{V_G}$ to group vertices in graph G . Each pair in $\mathcal{R}$ is in the form of (u, V) , where u is a query vertex in Q and V is a vertex set in G that are topologically equivalent. For example, given an edge (u_1, u_2) in Q and two pairs (u_1, V_1) and (u_2, V_2) in $\mathcal{R}$, either every vertex in V_1 is adjacent to each vertex in V_2 , or no edge exists between any vertex in V_1 and any vertex in V_2 . Thus set V can be treated as a vertex in subgraph matching (see Section 4).
- (2) We filter vertices that does not satisfy triangle relations during the computation of $\mathcal{R}$. Given a query vertex u of Q and a pair

$(u, V) \in \mathcal{R}$, each vertex $v \in V$ must satisfy the following: if u is in a triangle formed by u, u_1, u_2 in Q , then v is in a triangle formed by vertices v, v_1 and v_2 in G with $v_1 \in V_1, v_2 \in V_2, (u_1, V_1) \in \mathcal{R}$ and $(u_2, V_2) \in \mathcal{R}$.

3.1 Definition of Vertex-set Simulation

We present the definition of vertex-set simulation, and establish the uniqueness of the minimum maximal vertex-set simulation relation.

Vertex-set simulation. Given a query $Q = (V_Q, E_Q, L_Q)$ and a graph $G = (V_G, E_G, L_G)$, we say that G matches Q via *vertex-set simulation*, denoted by $Q \preceq_{vs} G$ if there exists a binary *match relation* $\mathcal{R} \subseteq V_Q \times 2^{V_G}$ satisfying the following conditions C1–C3.

(C1) Label consistency. For each $(u, V) \in \mathcal{R}$ and each $v \in V$, $L_Q(u) = L_G(v)$; i.e., each vertex in V has the same label as the query vertex u .

(C2) Edge consistency. For each $(u, V) \in \mathcal{R}$ and each query edge $(u, u') \in E_Q$, (a) there exists $(u', V') \in \mathcal{R}$ such that $(v, v') \in E_G$ for each $v \in V$ and $v' \in V'$; and (b) for any $(u', V') \in \mathcal{R}$, if there exists an edge $(v_0, v'_0) \in E_G$ for some $v_0 \in V$ and $v'_0 \in V'$, then each vertex $v \in V$ has an edge to each $v'' \in V''$, i.e., $(v, v'') \in E_G$ for each $v \in V$ and $v'' \in V''$.

(C3) Triangle preservation. For each $(u, V) \in \mathcal{R}$ and each query edge $(u_1, u_2) \in E_Q$ with $u_1, u_2 \in N_Q(u)$, i.e., u, u_1 and u_2 form a triangle in Q , there exist $(u_1, V_1), (u_2, V_2) \in \mathcal{R}$ such that some vertices $v \in V, v_1 \in V_1$ and $v_2 \in V_2$ form a triangle in G satisfying that $(v, v_1), (v, v_2)$ and (v_1, v_2) are edges in G .

Intuitively, (1) Condition (C1) enforces that for each $(u, V) \in \mathcal{R}$, query vertex u and all vertices in V carry the same label. (2) Condition (C2) enforces that *vertices in V have the same neighbors with respect to Q* . That is, for each edge (u, u') of query vertex u in Q , there exist $(u, V), (u', V') \in \mathcal{R}$ such that V is fully connected to V' , i.e., every vertex in V is adjacent to each vertex in V' ; moreover, given any $(u, V), (u', V') \in \mathcal{R}$, the edges between vertices in V and V' have the all-or-nothing property, i.e., either every vertex in V is adjacent to each vertex in V' or no edge exists between V and V' . (3) Condition (C3) preserves local triangle structures. Such constraint has been used in subgraph counting [23, 47].

Example 2: Consider query Q_1 and graph G_1 in Figure 1. We can verify that the following relation is a vertex-set simulation relation: $\mathcal{R} = \{(u_1, V_1), (u_2, V_2), (u_2, V_3), (u_3, V_4), (u_3, V_5)\}$, where $V_1 = \{v_1\}$, $V_2 = \{v_3, v_4\}$, $V_3 = \{v_5\}$, $V_4 = \{v_6, v_7, v_8\}$, and $V_5 = \{v_9\}$. Indeed, (1) $\mathcal{R}$ satisfies label consistency. For example, for (u_1, V_1) , u_1 and all vertices in V_1 share the label a . (2) $\mathcal{R}$ satisfies edge consistency. For query edge (u_2, u_3) , and pairs (u_2, V_2) and (u_3, V_4) , all vertices in V_2 has an edge to each vertex in V_4 ; but no edge exists between vertices in V_2 and V_5 for (u_3, V_5) . And (3) $\mathcal{R}$ satisfies triangle preservation. Vertices u_1, u_2 and u_3 form a triangle in Q_1 , and vertices v_1, v_3 and v_6 from the sets V_1, V_2 and V_4 , respectively, also form a triangle in G_1 .

There is more than one vertex-set relation for Q_1 on G_1 ; e.g., $\mathcal{R}' = \{(u_1, V_1), (u_2, V_2), (u_3, V_4)\}$ is also a vertex-set relation. $\square$

Property. While there may be multiple matches of Q in G via vertex-set simulation, there exists a unique *minimum maximal* match.

Maximal vertex-set simulation. Intuitively, a vertex-set simulation $\mathcal{R}$ is maximal, if it cannot be further extended by adding any vertex in G . To formalize this, for any query vertex $u \in V_Q$, let $C_{\mathcal{R}}(u) = \bigcup_{(u, V) \in \mathcal{R}} V$ denote the set of all vertices that can be matched by

u in vertex-set simulation. Then $\mathcal{R}$ is *maximal* if there exists no vertex-set simulation relation $\mathcal{R}'$, such that $C_{\mathcal{R}}(u) \subset C_{\mathcal{R}'}(u)$ for some $u \in V_Q$. Intuitively, the maximal condition ensures that $\mathcal{R}$ consists all vertices in G that can be matched by u .

Minimization. The maximal vertex-set simulation is not unique. Given a maximal vertex-set simulation $\mathcal{R}$, we can deduce another maximal vertex-set simulation as follows: (1) given $(u, V) \in \mathcal{R}$ with $|V| > 1$, replace (u, V) with (u, V_1) and (u, V_2) satisfying $V = V_1 \cup V_2$, $V_1 \neq \emptyset$ and $V_2 \neq \emptyset$. The resulting relation is also maximal, since all vertices matched by u are contained in the relation.

Among all maximal vertex-set simulation relations, we define a relation to be *minimum* if it has the smallest number of pairs. We show that such minimum relation is unique. See [1] for details.

Theorem 1: For query Q and graph G with $Q \preceq_{vs} G$, there is a unique minimum maximal vertex-set simulation relation $\mathcal{R}$ of Q in G . $\square$

Proof sketch: We prove it by contradiction. Assume that $\mathcal{R}_1$ and $\mathcal{R}_2$ are distinct minimum maximal vertex-set simulation relation of Q in G . We define a smaller relation $\mathcal{R}$ by merging vertex sets from $\mathcal{R}_1$ and $\mathcal{R}_2$ that have identical neighborhoods w.r.t. Q . More specifically, we say that (u, V_1) and (u, V_2) in $\mathcal{R}_1 \cup \mathcal{R}_2$ share the same neighbors with respect to Q , if for any neighbor u' of u in Q and any pair (u', V') , every vertex in V_1 has an edge to every vertex in V' if and only if every vertex in V_2 has an edge to every vertex in V' . Then we replace (u, V_1) and (u, V_2) by a pair $(u, V_1 \cup V_2)$. We can show that (1) $\mathcal{R}$ is a vertex-set simulation relation; (2) $\mathcal{R}$ is maximal; and (3) $\mathcal{R}$ is smaller than $\mathcal{R}_1$ and $\mathcal{R}_2$, i.e., $|\mathcal{R}| < |\mathcal{R}_1|$ and $|\mathcal{R}| < |\mathcal{R}_2|$. Therefore, $\mathcal{R}$ is a smaller maximal vertex-set simulation relations, a contradiction. Hence, the minimum maximal vertex-set simulation is unique. $\square$

Remark. (1) The uniqueness of the maximum relation has been shown for graph simulation [17, 28], dual simulation [41], strong simulation [41] and parametric simulation [18]. In contrast, maximal vertex-set simulations are not unique, since its maximality is defined by the number of vertices matched by query vertices, to ensure the soundness of the subgraph enumeration (see Section 4.1), rather than by the number of relation pairs.

(2) We aim to find a vertex-set simulation with the minimum number of pairs, since we will construct a contraction graph by representing each pair as a vertex, to accelerate the subgraph enumeration (see Section 4.2). Thus reducing the size of the vertex-set simulation relation leads to a smaller contraction graph and faster the subgraph enumeration.

3.2 Algorithm for Vertex-Set Simulation

Given a query Q and a graph G , we design Algorithm VSSim to compute the minimum maximal vertex-set simulation of Q in G .

Challenges. It is nontrivial to conduct the computation efficiently.

(1) For each pair $(u, V) \in \mathcal{R}$, how can we ensure that all vertices in V have the same identical neighbors with respect to Q . (2) Enumerating all triangles is costly [23], how can we avoid such enumerating when computing vertex-set simulation? (3) To compute minimum maximal vertex-set simulation, we cannot enumerate all candidate sets of vertices for each query vertex u , since there may exist exponentially many such sets.

We address these challenges as follows.

(1) We group vertices that have identical neighbors with respect to Q in the following two steps: (a) group vertices having the same identical neighbors; and then (b) iteratively update the grouping based on the topological structure of Q .

(a) To group vertices having the same neighbors, we encode the neighbors of each vertex as a string and group them by the hash values of these strings. More specifically, we first assign a distinct ID to each vertex. Then we encode the neighbors of vertex v in G as follows: collect all neighbors of v and order them based on their IDs to form a string. Then when two vertices have the same neighbors, their encodings of neighbors are the same. Therefore, we can group vertices with the same hash values.

(b) We next iteratively update the grouping. More specifically, if a pair (u, V) does not belong to any vertex-set simulation, e.g., if u has an edge to a vertex label λ , but no vertex in V has such a neighbor, then we update the encodings of vertices in the neighborhoods of V by removing IDs of vertices in V , and merge vertex sets whose encodings are the same. However, we only update the encodings and defer the merging of these sets after all invalid matches are removed, since merging vertex sets in each iteration is costly, which would degrade the performance of Algorithm VSSim.

(2) We exploit the triangle edges from [23] to filter candidate matches. Intuitively, the triangle edges of a vertex v are the edges between neighbors of v . If vertex v can match a query vertex u , then v has a triangle edge (v_1, v_2) that matches the triangle edge (u_1, u_2) of u , and v_1 (resp. v_2) is a candidate match of u_1 (resp. u_2).

(3) To compute all pairs $(u, V) \in \mathcal{R}$, we partition all vertices in G having the same label as u into multiple groups to form these pairs. In this way, we can not only avoid enumerating all subsets of vertices, but also ensure the vertex-set simulation is maximal and minimum. Indeed, since we partition candidate matches of u , i.e., all matches of u are included in $\mathcal{R}$, the computed relation must be maximal; and the relation is also minimum, since we merge vertex sets having the same neighbors w.r.t. Q .

Algorithm. Putting these together, we present Algorithm VSSim in Figure 3. Given a query graph Q and a graph G , it returns the minimum maximal vertex-set simulation relation $\mathcal{R}$ of Q in G .

Algorithm VSSim has the following three phases.

Phase 1: Initialization. Given Q and G , VSSim first groups vertices in G having the same labels and neighbors (lines 1-2). For each query vertex u , it collects all vertices in G having the same label as u , denoted by $C(u)$ (i.e., constraint C1; line 1). Then it partitions vertices in $C(u)$ into groups based on their edges (i.e., constraint C2(b); line 2). More specifically, two vertices v_1 and v_2 in $C(u)$ are in the same group K if for each query edge (u, u') and each vertex v' in $C(u')$, (v_1, v') is an edge in G if and only if (v_2, v') is an edge in G . Let $\mathcal{P}(u)$ be the set of all groups K for u .

Phase 2: Candidate match filtering. Algorithm VSSim then iteratively removes pairs (u, V) that violate the conditions of edge consistency (i.e., constraint C2(a)) or triangle preservation (i.e., constraint C3) of vertex-set simulation. More specifically, (1) it first checks the edge consistency (line 7). For pair (u, K) , it checks for each edge (u, u') in Q , whether each vertex in K has an edge to

Input: A query graph Q and A graph G .
Output: The minimum maximal vertex-set simulation $\mathcal{R}$.

```

1:  $C := \text{Group}(Q, G)$ ;
2:  $\mathcal{P} := \text{Partition}(Q, G, C)$ ;
3:  $\text{changed} := \text{true}$ ;
4: while  $\text{changed}$  do
5:    $\text{changed} := \text{false}$ ;
6:   for each  $u \in V_Q$  and each  $K \in \mathcal{P}(u)$  do
7:      $\text{bExistEdge} := \text{checkAdj}(Q, G, u, K)$ ;
8:      $\text{bTriangle} := \text{checkTri}(Q, G, u, K)$ ;
9:     if  $\text{bExistEdge} = \text{false} \vee \text{bTriangle} = \text{false}$  then
10:       $\mathcal{P}(u) := \mathcal{P}(u) \setminus \{K\}$ ;
11:       $\mathcal{P} := \text{UpdateSets}(\mathcal{P})$ ;
12:       $\text{changed} := \text{true}$ ;
13:  $\mathcal{R} := \emptyset$ ;
14: for each query vertex  $u \in V_Q$  do
15:    $\mathcal{P}_{\text{merge}}(u) := \text{MergeSet}(Q, G, \mathcal{P}(u))$ ;
16:   for each group  $K \in \mathcal{P}_{\text{merge}}(u)$  do
17:      $\mathcal{R} := \mathcal{R} \cup \{(u, K)\}$ ;
18: return  $\mathcal{R}$ ;
```

Figure 3: Algorithm VSSim

every vertex in K' for at least one set K' in $\mathcal{P}(u')$, if not, it removes K from $\mathcal{P}(u)$ (lines 9-10). (2) The algorithm next checks the triangle preservation (line 8). For each triangle edge (u_1, u_2) of u , it checks whether for each vertex v in K , there exists a triangle edge (v_1, v_2) of v such that $v_1 \in K_1$ and $v_2 \in K_2$ for some $K_1 \in \mathcal{P}(u_1)$ and $K_2 \in \mathcal{P}(u_2)$; if not, remove K from $\mathcal{P}(u)$ (lines 9-10), and updates the encodings of vertices having edges to vertices in K (line 11).

Phase 3: Candidate merging. Finally, Algorithm VSSim merges remaining groups for each query vertex. More specifically, given two groups K_1 and K_2 in $\mathcal{P}(u)$, K_1 and K_2 can be merged if they have the same neighborhood encoding (Line 15).

When no groups can be merged, constructs and returns the relation $\mathcal{R}$ as the minimum maximal vertex-set simulation (line 16-18).

Example 3: Algorithm VSSim computes the minimum maximal vertex-set simulation relation $\mathcal{R}$ of Q_1 in G_1 from Figure 1 as follows.

(1) In **Phase 1**, it first collects vertices having the same labels and obtains $C(u_1) = \{v_1, v_2\}$, $C(u_2) = \{v_3, v_4, v_5\}$, and $C(u_3) = \{v_6, v_7, v_8, v_9\}$. Then it groups vertices with the same neighbors as query vertices using hash values. Consider query vertex u_2 , candidates v_3 and v_4 share the same neighbors; indeed, both v_3 and v_4 have edges linking to vertices v_1, v_6, v_7 and v_8 . Although vertex v_5 also carries the label b , it links to vertices v_1, v_2 , and v_9 , which are different from neighbors of v_3 and v_4 . Therefore, Algorithm VSSim generates $\mathcal{P}(u_2) = \{\{v_3, v_4\}, \{v_5\}\}$ for query vertex u_2 .

(2) In **Phase 2**, it iteratively removes invalid matches. Consider $K = \{v_2\} \in \mathcal{P}(u_1)$. Vertex v_2 has the neighbors that are candidate matches of u_2 and u_3 , but it does not belong to any triangle in G_1 . Then $K = \{v_2\}$ is removed from $\mathcal{P}(u_1)$. Moreover, the algorithm updates the encoding of neighborhoods of v_5 and v_8 .

(3) Finally, in **Phase 3**, the algorithm merges vertex set having the same neighbors w.r.t. Q_1 . Consider the candidate matches v_6, v_7 and

v_8 of u_3 . Although they have different sets of neighbors, they are in the same group with respect to Q_1 . Indeed, (a) the neighbors of v_6 and v_7 are almost the same, except that v_6 and v_7 are adjacent to each other. However, u_3 does not have an edge linking to a vertex labeled c , then v_6 and v_7 cannot be in the same match of Q_1 in G_1 . Similarly, (b) after the removing $\{v_2\}$ from $\mathcal{P}(u_1)$, vertex v_8 has the same neighbors as v_6 and v_7 with respect to Q_1 , it merges these vertices into a set. Finally, the minimum maximal relation $\mathcal{R}$ is $\{(u_1, \{v_1\}), (u_2, \{v_3, v_4\}), (u_2, \{v_5\}), (u_3, \{v_6, v_7, v_8\}), (u_3, \{v_9\})\}$. $\square$

Correctness. We show that Algorithm VSSim correctly computes the unique minimum maximal vertex-set simulation relation $\mathcal{R}$.

(1) We first show that $\mathcal{R}$ is a vertex-set simulation. (a) for each $(u, V) \in \mathcal{R}$, each vertex $v \in V$ has the same label as u (line 1). (b) For each $(u, V) \in \mathcal{R}$ and each query edge $(u, u') \in E_Q$, (i) there exists $(u', V') \in \mathcal{R}$ such that $(v, v') \in E_G$ for each $v \in V$ and $v' \in V'$; otherwise, $(u, V) \in \mathcal{R}$ are removed from $\mathcal{R}$ (line 7); and (ii) for any $(u', V'') \in \mathcal{R}$, if there exists $v \in V$ and $v'' \in V''$ such that $(v, v'') \in E_G$, then each vertex in V has an edge to every vertex V'' ; indeed, two vertices are in the same set V if they have the same neighbors in G (line 2), or they are merged since they have the same topological structure as u (line 15). (c) For each $(u, V) \in \mathcal{R}$ and each vertex $v \in V$, if u and neighbors of u form a triangle in Q , then v has a triangle in G as u in Q (line 8).

(2) We next show that $\mathcal{R}$ is maximal. Assume by contradiction that $\mathcal{R}$ is not maximal. Then there exists another vertex-set simulation relation $\mathcal{R}'$, a query vertex u and vertex v such that $C_{\mathcal{R}}(u') \subset C_{\mathcal{R}'}(u)$ for $u \in V_Q$. However, if $C_{\mathcal{R}}(u') \subset C_{\mathcal{R}'}(u)$ and $v \in C_{\mathcal{R}'}(u) \setminus C_{\mathcal{R}}(u)$, then vertex v satisfies the label constraint (C1) and the topological structure constraint (C2) and (C3). Hence v cannot be removed from the set of candidates for u (lines 7-8). Then there exists $(u, V_1) \in \mathcal{R}$ satisfying $v \in V_1$, a contradiction.

(3) We finally show that $\mathcal{R}$ is minimum. Assume by contradiction that $\mathcal{R}$ is not minimum. Let $\mathcal{R}'$ be the unique minimum maximal vertex-set simulation relation of Q in G (see Theorem 1). Because $\mathcal{R}$ is maximal, there exist two pairs $(u, V_1), (u, V_2) \in \mathcal{R}$ and a pair $(u, V) \in \mathcal{R}'$ such that $V_1 \cap V \neq \emptyset$ and $V_2 \cap V \neq \emptyset$. Because $\mathcal{R}'$ be the unique minimum maximal vertex-set simulation relation, we know that all vertices in V_1 and V_2 have the same neighbors w.r.t. Q . Then V_1 and V_2 are merged in VSSim (line 15), a contradiction. Therefore, $\mathcal{R} = \mathcal{R}'$, i.e., $\mathcal{R}$ is minimum.

Complexity Analysis. VSSim takes $O(|V_Q||V_G|d_G \log d_G)$ time; here d_G (resp. d_Q) are maximum degrees of G (resp. Q).

The first phase takes $O(|V_Q||V_G|d_G \log d_G)$ time, since (1) grouping vertices with the same labels takes $O(|V_Q||V_G|)$ time; and (2) grouping vertices with identical neighbors takes $O(|E_Q||E_G|)$ time, since (a) collecting neighbors for each vertex takes $O(d_G)$ time; and (b) ordering them takes $O(d_G \log d_G)$ time.

Phase 2 takes $O(|V_Q||V_G|d_G^2 d_Q^2)$ time. Observe that (1) there exist at most $O(|V_Q||V_G|)$ elements; (2) checking neighbors for each element takes $O(d_G d_Q)$ time (line 7), since all vertices in the vertex sets have the same neighbors and it suffices to check only one vertex in the set; (3) checking triangles takes $O(d_G^2 d_Q^2)$ time due to the reason above (line 8); and (4) updating vertex-set simulation is in $O(d_G d_Q)$ time, since each element affects at most $O(d_G d_Q)$ elements.

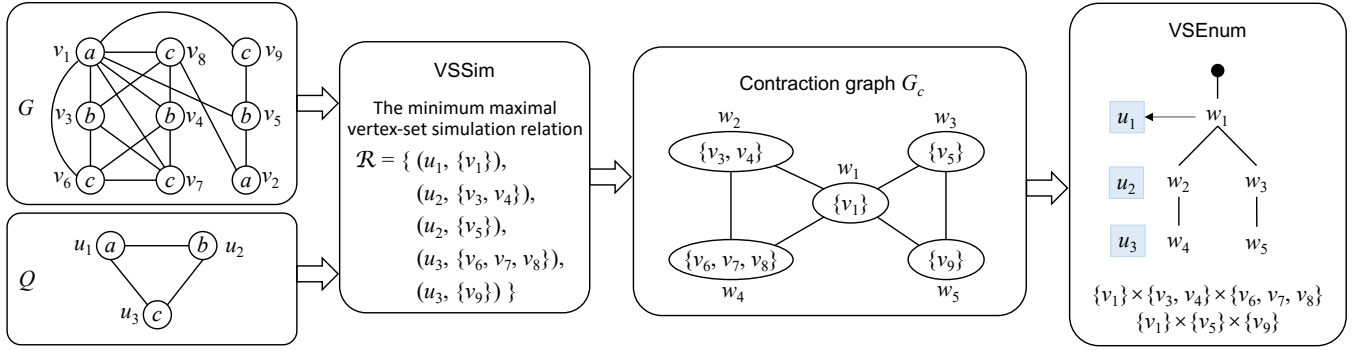


Figure 4: The workflow of VSMATCH. Given a query Q and a graph G , it computes the matches of Q in G as follows: (1) it first invokes VSSim to compute the vertex-set simulation $\mathcal{R}$ of Q on G ; (2) it next constructs a contraction graph G_c based on $\mathcal{R}$; and (3) it finally enumerates matches of Q in G using the partial matches of Q on G_c and the Cartesian product of vertex sets in $\mathcal{R}$.

Phase 3 takes $O(|V_Q||V_G|)$ time, since each query vertex can match at most $|V_G|$ vertex sets, and grouping sets with the same vertex takes $O(|V_G|)$ time by using Union-Find [14].

4 ALGORITHMS FOR SUBGRAPH MATCHING

In this section, we present Algorithm VSMATCH to enumerate subgraph matches based on vertex-set simulation.

Overview. As shown in Figure 4, given a graph G and a query Q , VSMATCH enumerates matches of Q in G in the following two steps.

(1) It first computes the minimum maximal vertex-set simulation relation $\mathcal{R}$ of Q in G , and then builds a contraction graph G_c based on $\mathcal{R}$. Intuitively, for each pair $(u, V) \in \mathcal{R}$, we treat set V as a vertex in the contraction graphs. Since vertices in V have the same neighbors w.r.t. Q , we can enumerate the subgraphs in a batch manner.

(2) It next enumerates the matches of Q in G by first enumerating its matches in G_c , and then expanding these matches into the matches in G . We can apply any existing subgraph enumeration algorithms for this. Here, we take IVE as a case study to show how to enumerate the matches of Q in G using contraction graphs.

In the following, we will first show how to construct the contraction graphs (Section 4.1), and then present the algorithm for subgraph enumeration (Section 4.2).

4.1 Graph Contraction

Given a minimum maximal vertex-set simulation $\mathcal{R}$, we construct a contraction graph G_c by encoding a set of vertices in G with a vertex, to compactly represent G and accelerate the enumeration.

More specifically, the contraction graph $G_c = (V_c, E_c, L_c)$ is defined as follows. (1) For each pair $(u, V_1) \in \mathcal{R}$, the set V_c contains a vertex v_{V_1} representing the set V_1 . (2) The edge set E_c is constructed according to the edges between represented vertex sets. Specifically, for any two vertices v_1 and v_2 in V_c , let V_1 and V_2 be the vertex sets represented by v_1 and v_2 with respect to (u_1, V_1) and (u_2, V_2) , respectively. We add an edge (v_1, v_2) to E_c if (u_1, u_2) is an edge in Q and there exists an edge between some vertex in V_1 and vertex in V_2 ; recall that if vertex v_1 in V_1 has an edge to vertex v_2 in V_2 , then every vertex in V_1 has an edge to v_2 due to the definition of vertex-set simulation. (3) For each vertex $v_{V_1} \in V_c$, $L_c(v_{V_1})$ is the

label of any vertex in V_1 ; recall that all vertices in V_1 carry the same label due to the definition of vertex-set simulation.

Example 4: Consider query Q_1 and graph G_1 in Figure 1. The minimum maximal vertex-set simulation relation $\mathcal{R}$ for Q_1 on G_1 is computed as follows (see Example 2): $\mathcal{R} = \{(u_1, V_1), (u_2, V_2), (u_2, V_3), (u_3, V_4), (u_3, V_5)\}$, where $V_1 = \{v_1\}$, $V_2 = \{v_3, v_4\}$, $V_3 = \{v_5\}$, $V_4 = \{v_6, v_7, v_8\}$, and $V_5 = \{v_9\}$.

The contraction graph is constructed as follows: (1) each set V_i ($i \in [1, 5]$) is represented by a vertex w_i ; (2) the edge set is defined as follows: when there exists an edge from vertex in V_i to a vertex in V_j , there exists an edge between w_i and w_j in the contraction graph; for example, there exists an edge between w_1 and w_2 , since v_1 has edges linking to v_3 and v_4 , i.e., vertices in V_2 . (3) the labels of the vertices are the same as the ones in the vertex sets; vertex w_2 is b , since both v_3 and v_4 in V_2 carry the label b .

There exist two triangles in G_c that match Q_1 . From the two triangles, we can easily identify seven triangles matching Q_1 , one for w_1, w_3, w_5 and six for w_1, w_2, w_4 , since w_2 has two candidate matches and w_4 has three candidate matches. Using the contraction graph, we can enumerate all matches of Q_1 in G_1 by applying Cartesian products on the matches of Q_1 on G_c . $\square$

We show that G_c is sound for enumerating all matches of Q in G , i.e., we can enumerate all matches of Q in G using only the contraction graph G_c without accessing the original graph G [7].

Theorem 2: Given a graph G and a query Q , if M is a match of Q in G , there exists a match M_c of Q in the contraction graph G_c such that for each vertex $M(u)$ in G with $u \in V_Q$, vertex $M(u)$ also exists in $V_{M_c(u)}$, where $M(u)$ (resp. $M_c(u)$) is the vertex matched by u in G (resp. G_c), and $V_{M_c(u)}$ is the vertex set represented by $M_c(u)$ in G_c . $\square$

PROOF. Given M , since $\mathcal{R}$ is a maximal vertex-set simulation relation, for each vertex $M(u)$, there is a pair $(u, V_u) \in \mathcal{R}$ such that $M(u) \in V_u$; otherwise, we can extend $\mathcal{R}$ with $(u, M(u))$, which contradicts the fact that $\mathcal{R}$ is maximal. Let $u_1, \dots, u_n$ be all vertices in Q , and $v_{u_1}, \dots, v_{u_n}$ be the vertices in G_c representing $V_{u_1}, \dots, V_{u_n}$, respectively. We show that $v_{u_1}, \dots, v_{u_n}$ forms a match of Q in G_c . Then M can be deduced from the match formed by $v_{u_1}, \dots, v_{u_n}$.

To this end, we prove the following two properties: (1) for each vertex u_i ($i \in [1, n]$), v_{u_i} carries the same labels as u_i ; and (2) for each edge between u_i and u_j in Q , there exists an edge between

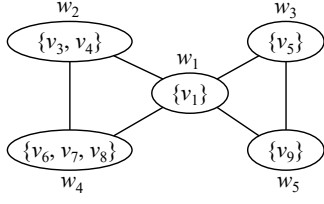


Figure 5: The contraction graph for Q on G .

v_{u_i} and v_{u_j} in G_c . For (1), because $M(u_i) \in V_{u_i}$, all vertices in V_{u_i} carry the same label as u_i . From the definition of G_c , we have that $L_c(u_i) = L(u_i)$. For (2), for the edge between u_i and u_j in Q , because M is a match Q in G , there exists an edge between $M(u_i)$ and $M(u_j)$. Because $\mathcal{R}$ is a maximal vertex-set simulation, there exists an edge between each vertex $v_i \in V_i$ and each vertex $v_j \in V_j$. There exists an edge between v_{u_i} and v_{u_j} in G_c due to the construction of G_c . $\square$

Remark. (1) Given a match M_c of Q in G_c , since all vertices in $V_{M_c(u)}$ have the same neighbors, we can enumerate the matches of Q in G by taking the Cartesian product of the sets $V_{M_c(u)}$ (see Section 4.2).

(2) When G is large, it is costly to compute the minimal maximal vertex-set simulation. We can compute a few rounds of VSSim to strike a balance between efficiency and effectiveness. In Section 5, 2 rounds suffice to filter most invalid matches; see also [24].

4.2 Subgraph Enumeration

Given the contraction graph G_c , we extend existing algorithm IVE [31] to enumerate all matches of Q in the original graph G .

Review of Algorithm IVE. We start with a review of Algorithm IVE. Its key idea is to divide the vertices of Q into isolated and non-isolated vertices, and then enumerate the matches of Q in G by computing the Cartesian product of the matches of these two parts [31].

More specifically, IVE consists of the following two steps. (1) It first partitions the vertices of Q into non-isolated and isolated vertices by iteratively removing a vertex with maximum degree from the query. After all edges are removed, the remaining vertices and the removed vertices are treated as isolated and non-isolated vertices, respectively, and the removal order of non-isolated vertices forms a matching order $\mathcal{O}$ for Q . (2) It then enumerates the matches of the non-isolated vertices by backtracking, following the matching order $\mathcal{O}$. When all non-isolated vertices have been matched, the full matches of Q are computed by taking the Cartesian product between the partial matches of the non-isolated vertices and isolated vertices. Here, the partial matches of isolated vertices are the vertices in G carrying the same labels as isolated query vertices.

Challenges. To extend IVE to enumerate the matches of Q in G using contraction graphs, we need to address the following challenges.

- (1) When determining the matching order, if multiple query vertices have the same degree, IVE selects the one with the smallest candidate set. However, in the contraction graph, different vertices may represent different numbers of vertices in the original graphs. Can we deduce a better matching order by using such information.
- (2) The matches of Q in G can be generated from the matches of Q

Input: A graph $G = (V, E, L)$, query $Q = (V_Q, E_Q, L_Q)$

Output: All matches of Q in G .

- 1: $G_c := \text{BuildContraction}(Q, G)$;
- 2: $\mathcal{O} := \text{EstOrder}(Q, G_c)$;
- 3: $M_p := \emptyset$; $\text{GC} = \emptyset$;
- 4: $\text{Output} := \text{VSEnum}(G_c, G, M_p, \mathcal{O}, \text{GC})$;
- 5: **return** Output;

Figure 6: Algorithm VSMATCH

in the contraction graph G_c . More specifically, for each match of Q in G_c , we compute the Cartesian product of vertex sets represented by the matched vertices in G_c . However, not every tuple in this Cartesian product corresponds to a match of Q in G , since a vertex in G may appear in multiple vertex sets represented by different vertices in G_c . Under subgraph isomorphism, the match must be injective; that is, the same vertex in G cannot be matched by two query vertices in Q . How can we correctly enforce the isomorphic semantics when expanding the matches returned by IVE on G_c .

(3) To enumerate the matches on the original graphs, we compute the Cartesian-product of matches of isolated-vertex and non-isolated vertex matches. However, such computation is costly when the number of isolated vertices is larger. Moreover, if we defer the verification of isolated vertices to the end of the enumeration, the matches of the non-isolated vertices may be large, which results in redundant computation. Can we interleave the verification of isolated and non-isolated vertices to accelerate the enumeration?

We adopt the following strategies to address these issues.

(1) For the matching order, we always select a query vertex with the smallest candidate set in the contraction graph G_c . When multiple vertices share the same minimum candidate set, we break ties by choosing the one with the maximum degree in G_c . Such matching order results in better performance than the original matching order determined by IVE (see Section 5). This is because the vertex-set simulation has initially filtered out the candidates that cannot form a match, and the size of the candidate set can be a better indicator for searching space than the degree of u .

(2) We enumerate the matches of Q in G in the following three steps: (a) enumerate the matches of Q in the contraction graphs G_c ; (b) for each matched vertex v_c in G_c , replace v_c with the vertex set V represented by v_c ; and (c) enumerate the matches of Q in G by computing the Cartesian products of these vertex sets; during this process, we discard tuples that maps different query vertices to the same vertex in G , to enforce the isomorphic semantics.

(3) We introduce a notation, namely trigger points, to indicate when Cartesian products should be computed. More specifically, the trigger point of a query vertex u in V_Q , denoted by $\text{trig}(u)$, is defined as follows: (a) if u is a non-isolated vertex, then its trigger point is itself, i.e., $\text{trig}(u) = u$; otherwise, (b) when u is an isolated vertex, $\text{trig}(u)$ is the neighbor of u that appears last in the matching order $\mathcal{O}$. Intuitively, when the enumeration reaches $\text{trig}(u)$, all neighbors of the isolated vertex u have already been matched. We also maintain a global cache to record the results of these Cartesian products, which are later used to construct the full matches of Q in G .

Input: Contraction graph G_c , query Q , partial match M_p , order O and the cached matches GC of isolated vertex;

Output: All matches of Q in G extended from M_p .

```

1: if IsValid( $M_p$ ) then
2:   return CompleteMatch( $G_c, Q, M_p, GC$ );
3:  $u_i := \text{GetNextVertex}(M_p, O)$ ;
4: output :=  $\emptyset$ ;
5: for each vertex  $v_V$  in  $G_c$  such that  $(u, V) \in \mathcal{R}$  do
6:   if canExtend( $G_c, Q, M_p, u, v_V$ ) then
7:      $M'_p := M_p \cup \{(u_i, v_V)\}$ ;
8:     if  $u_i \in \text{trig}(u)$  for some isolated vertex  $u$  then
9:       cache := ExtendWithCar( $G_c, Q, M'_p, O, GC$ );
10:      GC := GC  $\cup$  cache;
11:      output := output  $\cup$  VSEnum( $G_c, Q, M'_p, O, GC$ );
12: return output;

```

Figure 7: Algorithm VSEnum

Algorithm. Putting these together, we present Algorithm VSMATCH in Figure 6. Given a graph $G = (V, E, L)$ and query $Q = (V_Q, E_Q, L_Q)$, it enumerates all matches of Q in graph G , i.e., all subgraphs of G isomorphic to Q . It first constructs the contraction graph G_c from G (line 1; see Section 4.1). It then establishes the matching order for vertices in Q (line 2). More specifically, it partitions Q into isolated vertices and non-isolated vertices as mentioned above, and deduces a matching order O for non-isolated vertices. It next initializes the partial matches M_p and the cached matches for isolated vertices GC with $\emptyset$ (line 3). After that, it enumerates all matches following the matching order O via procedure VSEnum (line 4).

Procedure VSEnum. Given graph G_c , query Q , partial match M_p , matching order O , and the cached matches GC of isolated vertex, VSEnum recursively enumerates all matches of Q in G by extending M_p (see Figure 7). It first checks whether all non-isolated vertices are matched in M_p (lines 1-2); if so, it extends M_p to generate full matches of Q in G by computing the Cartesian product of M_p and GC. Moreover, VSEnum further removes such invalid matches during computing the Cartesian product, e.g., the matches mapping two different query vertices to the same vertex in G . Otherwise, when there exists some non-isolated vertex in Q that has not been matched, VSEnum extends M_p with query vertex u_i following the matching order O (lines 3-11). For each candidate match v_V of u_i in the contraction graph G_c (i.e., (u_i, V) is a pair in the vertex-set simulation $\mathcal{R}$ of Q on G), VSEnum first checks whether v_V can be extended with M_p by inspecting the adjacent edges of v_V (line 6); if so, it extends M_p with the match (u_i, v_V) . Moreover, if u_i is a trigger point for isolated vertices $u_j^1, \dots, u_j^k$, then it computes the matches of these vertices using Cartesian product. It also caches the results for reuse (line 10). After that, it recursively checks other query vertices following the order O (line 11).

Example 5: Given query Q_1 and graph G_1 in Figure 1, Algorithm VSMATCH enumerates all matches of Q_1 on G_1 as follows:

(1) It first constructs the contraction graph as shown in Figure 5, and then establishes a matching order O as follows: u_1, u_2 . Vertex u_3 is an isolated vertex, and its trigger point is u_2 .

Table 2: Statistics of Real-world graphs

Dataset	$ V $	$ E $	d	Domain
yeast	3,101	12,519	8.1	Biological
human	4,674	86,282	36.9	Biological
hprd	9,303	34,998	7.5	Biological
email	36,692	183,831	10.0	Social
wordnet	146,005	656,999	9.0	Lexical
twitch	168,114	6,797,557	80.9	Social
dblp	317,080	1,049,866	6.6	Co-authorship
eu2005	862,664	16,138,468	37.4	Web
patents	3,774,768	16,518,947	8.8	Citation

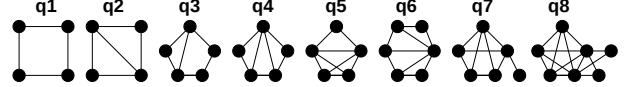


Figure 8: Global queries.

(2) It then enumerates the matches of Q_1 in G_c following the matching order O . Algorithm VSMATCH starts the enumeration by mapping u_1 to the unique vertex w_1 labeled a in G_c , and extends the partial match with neighborhoods of w_1 in G_c . Assume that in the next iteration, the partial match M_p maps u_2 to vertex w_2 .

(3) Since u_2 is the trigger point of the isolated vertex u_3 , given partial match M_p it computes the matches of Q_1 in G_1 by Cartesian product of the partial match M_p and candidate matches of u_2 and u_3 . That is, $\{v_1\} \times \{v_3, v_4\} \times \{v_6, v_7, v_8\}$, which results in six matches of Q_1 in G_1 .

(4) After that VSEnum backtracks to inspect another partial match M'_p that maps u_1 and u_2 to w_1 and w_2 , respectively. And VSMATCH terminates when all matches of Q_1 in G_c are generated. $\square$

Complexity. We show that VSMATCH retains the same complexity as IVE. Observe that (1) establishing matching order takes $O(|V_Q||V_G|)$ time, since there exist $|V_Q|$ query vertices and each query vertex has at most $|V_G|$ candidates; (2) the cost of VSEnum includes the time to deduce the matches of Q in G_c and compute the Cartesian product of vertex sets for the partial matches. Then the complexity of VSEnum is comparable to that of IVE, since the contraction graph G_c is smaller than G , and the vertex sets in the vertex-set simulation are not larger than the candidate sets of the isolated vertices in IVE.

5 EXPERIMENTS

Using real-life and synthetic graphs, we evaluate the effectiveness and efficiency of the proposed method.

Experimental setting. We start with the setting.

Graphs. We conduct experiments using both real-world and synthetic graphs. (1) We select nine real-world graphs widely used in previous subgraph matching studies [30, 49, 53] (see their statistics in Table 2). These graphs cover various domains, including biological networks, co-authorship networks, web graphs, and social networks. (2) To evaluate scalability, we generated synthetic graphs with EvoGraph[45], with up to 2M vertices and 42M edges.

Both real-world and synthetic graphs followed the same label assignment procedure. The number of vertex labels were 15, 30, 45, and 60, and vertex labels were randomly assigned [40, 49].

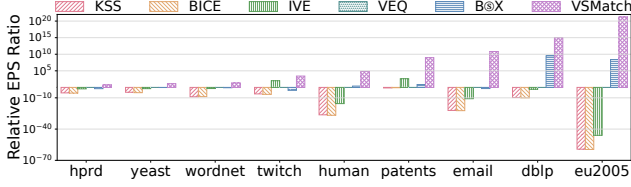


Figure 9: Relative EPS ratio (take VEQ as baseline method).

Queries. We generated queries using a sampling method [30]. More specifically, we performed a Metropolis-Hastings random walk [26] on graphs and extracted induced subgraphs as queries. By default, the query size was set to 10, 20, 30, 40, and 50. For each query size i , we generated a query set Q_i with 300 queries.

To evaluate the robustness of algorithm VSMATCH, we included challenging queries [32] (see Figure 8), *a.k.a.* global queries [40].

Algorithms. We implemented algorithm VSMATCH in C++. We also implemented the variant VSMATCH_{notri}, which disables the trigger point mechanism, VSMATCH_{ori}, which uses the candidate size in G to determine the matching order, and VSMATCH_{General}, which conducts backtracking on the original graph G . We compared with five baselines: (1) VEQ [35], which uses dynamic equivalences for backward-looking local pruning, (2) KSS [51], which reduces backtracking overhead by decomposing query vertices into conditional kernel and shell structures; (3) BICE [12], which compresses the search space through global bipartite matching with cell-wide verification; (4) IVE [31], which accelerates enumeration by separating isolated query vertices and resolving them via delayed bipartite matching; and (5) B⊗X [40], which groups vertices into search boxes for batch processing. We used the C++ implementations released by the original authors.

Metrics. We evaluate all methods using the following metrics.

(1) *Embeddings per second (EPS).* EPS measures the average number of embeddings (*i.e.*, matches) enumerated per second. We used EPS as a throughput metric to evaluate performance, incorporating both the number of generated embeddings and elapsed time [53].

(2) *Relative EPS Ratio.* We report the relative EPS ratio to compare throughput gains [40]. The EPS of each method was normalized by the EPS of VEQ [35]. Thus, the relative EPS ratio of VEQ was fixed to 1, and a larger ratio indicates higher enumeration throughput.

(3) *Query processing time (QPT).* QPT measures the time required to process a query and output all matches by default. In the scalability experiments (*i.e.*, Exp-2) with output limits, QPT measures the time required to output the specified number of matches.

(4) *Number of timed-out queries (NTQ).* NTQ measures the number of queries that fail to finish within the 3600s time limit.

Experiment Environment. All experiments were conducted on a Linux server running Ubuntu 24.04, with an Intel Xeon Silver 4214 CPU at 2.2 GHz and 256 GB of RAM. To strike a balance between efficiency and effectiveness, we run Algorithm VSSim for only 2 rounds to filter invalid matches, unless stated otherwise. Each experiment was repeated three times, and the average is reported.

Experimental results. We next report our findings.

Exp-1: Efficiency. We test the efficiency of VSMATCH.

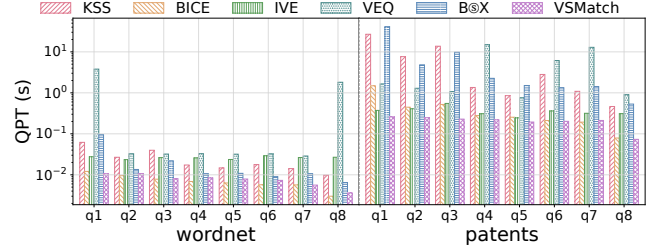


Figure 10: QPT of global queries.

Under the relative EPS metric. We evaluated the throughput of VSMATCH using the relative EPS ratio normalized by VEQ. The results are shown in Figure 9, and we find the following.

(1) Algorithm VSMATCH achieves the highest relative EPS ratio on all datasets. VSMATCH outperforms B⊗X by 16× on yeast, up to $10^{12} \times$ on eu2005, with an average speedup of nearly $10^6 \times$; similarly, it outperforms VEQ, KSS, BICE and IVE by $10^7 \times$, $10^{23} \times$, $10^{23} \times$ and $10^{15} \times$, respectively, on average.

(2) Algorithm VSMATCH achieves better performance on larger or denser graphs. VSMATCH beats B⊗X by $10^3 \times$ on dblp, up to $10^{12} \times$ on eu2005, with an average speed up of 10^8 ; it beats VEQ by $10^3 \times$ on twitch, up to $10^{21} \times$ on eu2005, with an average speed up over $10^{10} \times$. This is because VSMATCH enumerates on contraction graphs, which substantially reduces redundant exploration.

Global Query Evaluation. We evaluated the performance of VSMATCH using the challenging queries in Figure 8 on wordnet and patents with a timeout of 3600s. As shown in Figure 10, (1) on the small graph wordnet, most methods finish quickly due to the small search space. (2) On the large graph patents, VSMATCH outperforms KSS, BICE, IVE, VEQ and B⊗X by 33.49×, 2.12×, 1.76×, 24.21× and 38.42×, respectively, since the enumeration over contraction graphs reduces redundant exploration.

Exp-2: Scalability. We evaluate the scalability of VSMATCH by varying query size, label size, graph size and output limit.

Varying the query size. We vary the query size $|Q|$ from 10 to 50. For each query size, we sample 200 queries and enumerate all full matches with a 3600s timeout. As shown in Figures 11(a) and (b), (1) as the query size increases, the running time of all algorithms increases, as expected. (2) VSMATCH has the lowest processing time and maintains the highest completion rate across all query sizes. When $|Q| = 50$, VSMATCH outperforms KSS, BICE, IVE, VEQ and B⊗X by 1.19×, 1.16×, 1.08×, 1.14× and 1.19×, respectively, on patents. This is because VSMATCH enumerates all matches on the contraction graphs, and when the query size increases, and the vertex-set simulation can filter more invalid candidates, which significantly reduces the searching space.

Varying the Label Size. We vary the label size L from 10 to 60. For each label size we sample 200 queries and enumerate all exact matches under a 3600s timeout. As shown in Figures 11(c) and (d), (1) when the label size L increases, all algorithms become faster, as expected, since the number of candidate matches for the queries becomes smaller with L increasing, which reduces the searching space. Especially, when L is large (*e.g.*, $L = 50$) most baselines finish in one second. (2) VSMATCH is faster than all baselines when L is

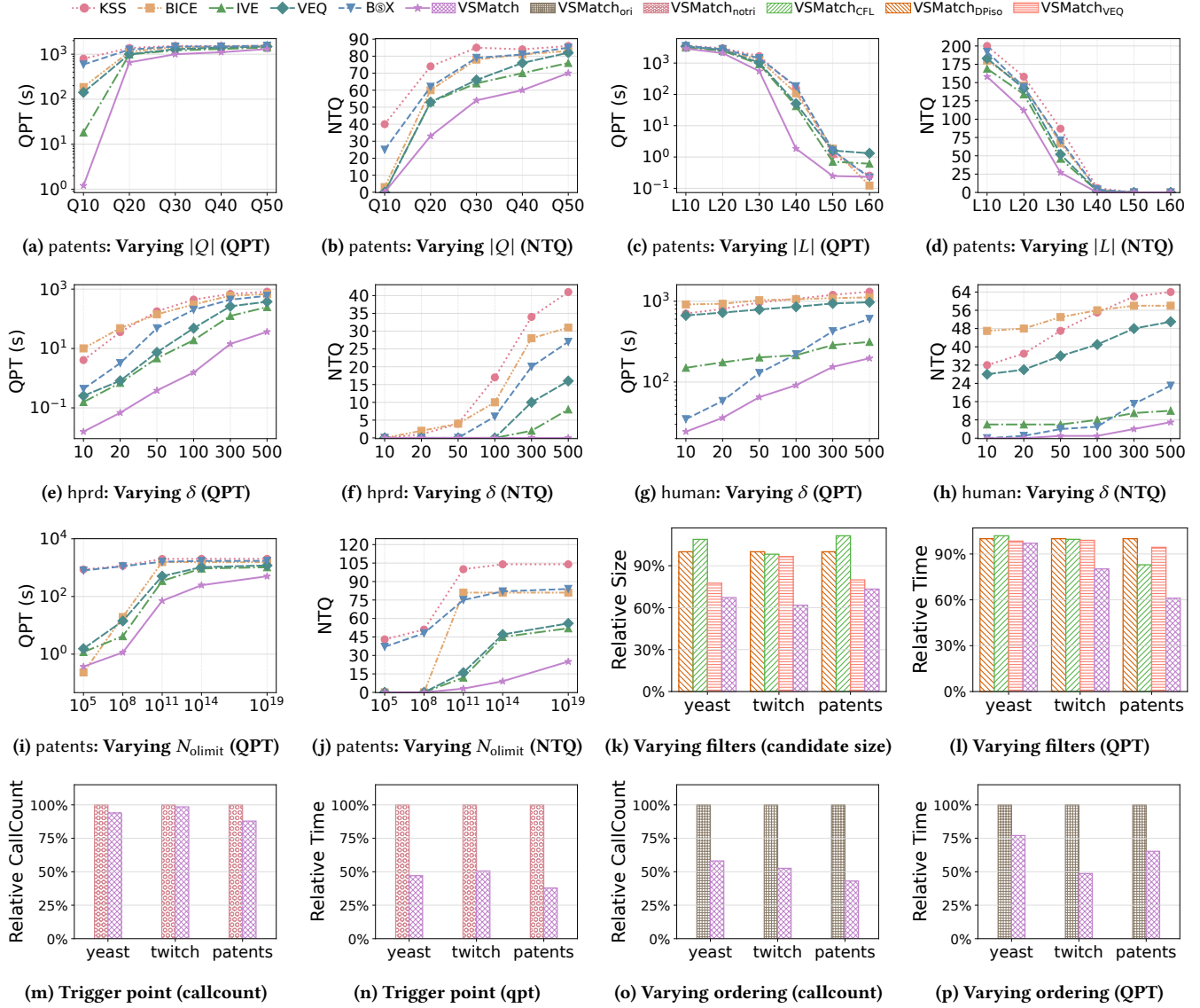


Figure 11: Performance evaluation

small. When $L = 10$, VSMATCH outperforms KSS, BICE, IVE, VEQ and $B \otimes X$ by 1.25 $\times$, 1.16 $\times$, 1.09 $\times$, 1.18 $\times$ and 1.22 $\times$, respectively, on patents. This is because VSMATCH utilizes vertex-set-level compression, and when L is small, the number of candidate matches for each query vertex is large, which enables VSMATCH to compress the graph more effectively, and reduces the searching space.

Varying Graph Size. We vary the graph size by up-scaling the original graphs using EvoGraph [45], which adds edges to the graphs with a preferential attachment mechanism and preserves the original topological properties. We scale two real-world datasets (hprd and human) by setting the scaling factors δ to 10, 20, 50, 100, 300, and 500. For each graph size, we sample 200 queries of size 10 from origin graph for each dataset. We then evaluate the QPT to enumerate all matches under a timeout of 3600s. As shown in Figure 11(e)–(h), (1) when the graph size grows, all algorithms get

slower, as expected. (2) On hprd-500 (3M vertices and 17M edges), VSMATCH has no timeouts and completes all queries with average processing time of 36.59s. (3) On dense human-500 (2M vertices and 42M edges), VSMATCH times out on only 7 queries, with average processing time of 196.91s, faster than $B \otimes X$, VEQ, KSS, BICE and IVE by 3.07 $\times$, 4.93 $\times$, 6.66 $\times$, 5.67 $\times$ and 1.59 $\times$, respectively.

Varying output limits. We next evaluated the robustness in terms of QPT by varying the output limits (N_{limit}) over 10^5 , 10^8 , 10^{11} , 10^{14} and 10^{19} , denoted as $o5$, $o8$, $o11$, $o14$ and $o19$, respectively. As shown in Figure 11(i)–(j), (1) when the limits increase, all algorithms take more time to enumerate the matches, as expected. (2) VSMATCH exhibits more stable performance than baselines at higher output limits. When the output limits vary from 10^5 to 10^{19} , the average processing time of VSMATCH increases by only 501.34s (from 0.37s to 501.71s), while the other baselines experience increases of

Table 3: Statistics of Contraction graphs

Dataset	$ V $	$ E $	$ V_c $	$ E_c $
yeast	3,101	12,519	19	23
twitch	168,114	6,797,557	21,686	108,680
patents	3,774,768	16,518,947	4,194	5,241

924.75s-1547.19s. This is because the contraction graphs reduce the searching space, which increases the throughput of VSMatch.

Exp-3: Ablation Study. We conducted ablation studies to evaluate the effectiveness of components in VSMatch, including VSSim filtering, enumeration, trigger points, and ordering.

Effectiveness of VSSim Filtering. We compared VSSim-based filtering with DPiso[24], CFL[8] and VEQ[35] using relative candidate size and relative query processing time ratio. We construct the contraction graphs using the candidate sets generated by these methods, and then enumerate all matches using VSMatch. The resulting variants are denoted as VSMatch_{CFL}, VSMatch_{VEQ}, and VSMatch_{DPiso}. Both candidate size and query processing time ratio are normalized by that of VSMatch_{DPiso}. As shown in Figure 11(k) and Figure 11(l), (1) VSSim generates the smallest candidate sets on all three datasets, resulting in the lowest relative query processing time ratio. The graphs generated by VSSim are 37.16%, 38.26%, 36.05% smaller than those generated by VSMatch_{CFL}, VSMatch_{VEQ}, and VSMatch_{DPiso}, respectively on twitch. (2) On the large graphs patents and denser twitch, VSSim can prune a large portion of the search space by using topological constraints (e.g., the triangle constraint), leading to lower processing time. On twitch, VSSim outperforms variants VSMatch_{CFL}, VSMatch_{VEQ} and VSMatch_{DPiso} by 1.26 $\times$, 1.25 $\times$ and 1.23 $\times$, respectively.

Table 3 also lists the average sizes of contraction graphs. We found that the contraction graphs are much smaller than the original graphs. On patents, the contraction on average contains only 0.1% as many vertices and 0.03% as many edges as the original graph.

We evaluated the impact of rounds of VSSim on performance of filtering. We found the following (not shown): (1) when the number of rounds gets larger, the contraction graph gets smaller, since more invalid matches are removed. (2) A few rounds of VSSim suffices to filter most invalid candidates. On patents, only 2 rounds of VSSim suffices to filter 99.73% of invalid candidates that can be found by VSSim upon termination; and VSSim terminates in 8 rounds.

Effectiveness of searching space pruning. To evaluate the pruning capability of VSMatch, we measured its search space size by varying at different recursive depths and compared it with a variant conducting backtracking on the original graph G , denoted by VSMatch_{General}. Using a sample of 100 queries of size 10, we recorded the average number of vertices visited during backtracking with recursion depths varying from 1 to 9 in Figure 12. We found the following: (1) when the recursion depths gets larger, the number of visited vertices gets larger, as expected; the average visited vertices of VSMatch gets smaller when the recursion depth is larger than some threshold, this is because the recursion depth of VSMatch is bounded by the non-isolated vertices in the queries. For example, on patents 3%, 15%, 40%, 78%, 100% queries have 5, 6, 7, 8, 9 non-isolated vertices, respectively. and (2) VSMatch visits the fewest number of vertices. On patents, VSMatch visits only

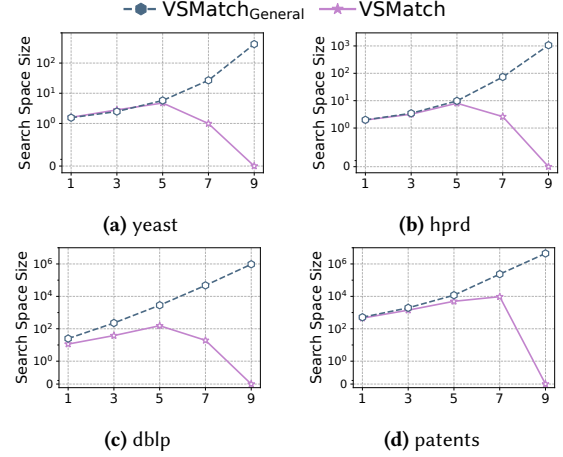


Figure 12: Search space size at different depths.

9,563 vertices on average, while VSMatch_{General} visits 235,189 vertices when the recursive depth is 7, since VSMatch_{General} conducts backtracking on the original graph.

Effectiveness of trigger points. We next evaluated the impact of trigger points. We compare VSMatch with the variant VSMatch_{notri}, which computes the Cartesian product only when all non-isolated vertices are matched and disable the global cache mechanism. We executed 200 random queries under default settings on three datasets (yeast, twitch, and patents). Performance is measured using the relative call count and relative query processing time, both normalized by that of VSMatch_{notri}. Here, *Call Count* refers to the total number of recursive function invocations during enumeration, measuring the size of the search space. As shown in Figure 11(m)–(n), (1) the trigger points can reduce the running time. VSMatch is 2.13 $\times$, 1.98 $\times$ and 2.65 $\times$ faster than VSMatch_{notri} on yeast, twitch, and patents, respectively. This is because trigger points can reduce invalid partial matches earlier, which reduces the search space and accelerate VSMatch. (2) On the twitch dataset, the relative processing time is substantially reduced, due to the trigger points and global cache.

Effectiveness of Adaptive Ordering. We next evaluated the impact of matching order on the performance of VSMatch. We compared VSMatch with the variant VSMatch_{ori}, which orders non-isolated vertices by their degrees in ascending order, and uses the candidate set sizes in G as a tie-breaker. Using a sample of 200 queries, Figure 11(o)–(p) report the relative execution time of VSMatch, normalized by that of VSMatch_{ori}. We found the following. (1) Our adaptive ordering strategy consistently outperforms the baseline across all three graphs, demonstrating that contraction graph can effectively guide the matching order to reduce the search space and improve enumeration efficiency. (2) On the dense twitch and large-scale patents graphs, VSMatch is 2.05 $\times$ and 1.53 $\times$ faster than the baseline, respectively. This is because the graphs are dense or larger, the vertex sets in the vertex-set simulation are large, and VSMatch can deduce a better matching order based on the candidate size. (3) On the sparse graph yeast, the performance of VSMatch and VSMatch_{ori} are similar, i.e., VSMatch is only 1.30 $\times$ faster VSMatch_{ori}, since vertex sets in the vertex-set simulation are small, the matching order has limited impact on the running time of the enumeration process VSEnum.

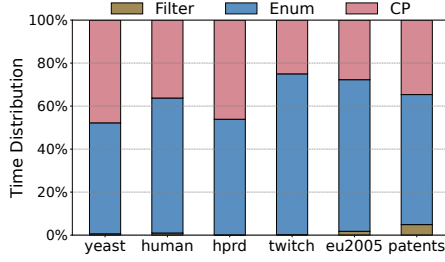


Figure 13: Time Decomposition.

Cost breakdown analysis. To understand the computational overhead of VSMatch, we decompose its execution time into three phases: filtering (Filter) using vertex-set simulation, enumeration (Enum), and Cartesian product (CP). Using a sample of 200 queries, Figure 13 shows the results on yeast, human, hprd, twitch, eu2005, and patents. We found the following. (1) The filtering phase is efficient, accounting for less than 5% of the overall execution time across all datasets (e.g., $\sim 0.60\%$ on yeast; $\sim 1.77\%$ on eu2005; and $\sim 4.89\%$ on patents). (2) The enumeration dominates the overall cost, as expected, and accounts for 52% to 75% of the total time. (3) Cart contributes around 25% to 48% of the execution time, since it exploits the Cartesian product to generate the matches, which is costly when a vertex in contraction graphs denotes a large number of candidates. For example, it accounts for $\sim 34.64\%$ of the running time of VSMatch on patents, since its contraction graph is much smaller than the original graph (see Table 3), i.e., each vertex in the contraction graph denotes a large number of vertices in the original graph, resulting in longer running time for Cartesian product.

Summary. From the experiments, we find the following. (1) VSMatch on average outperforms state-of-the-art algorithms B $\otimes$ X, VEQ, KSS, BICE and IVE by $10^6\times$, $10^7\times$, $10^{23}\times$, $10^{23}\times$ and $10^{15}\times$, respectively. (2) Vertex-set simulation is effective. Given a graph G with 3.8M vertices and 16.5M edges, it can effectively filter invalid candidates, and results in a contraction graph with 4194 vertices and 5241 edges. (3) Algorithm VSSim is efficient. It takes only 0.3s to process a graph with 3M vertices and 16M edges, and accounts for 4.89% of overall time of VSMatch. (4) VSMatch scales well. It takes only 196.91s to enumerate all matches of a query with 10 vertices on a graph with 2M vertices and 42M edges.

6 RELATED WORK

We categorize related work as follows.

Subgraph matching. Existing exact subgraph matching algorithms can be classified into two categories: join-based approaches [34, 39, 43] and backtracking-based approaches [10, 33]. (1) Join-based methods formulate the subgraph isomorphism problem as a multi-way relational join query over edge tables, e.g., EmptyHeaded [2] and BiGJoin [5]. (2) Most recent algorithms for subgraph matching adopt the backtracking-based paradigm, which incrementally maps query vertices to data vertices. The classical algorithm developed in [50] introduced the fundamental backtracking framework for subgraph isomorphism. Algorithm VF2 [13] introduced neighborhood feasibility rules for earlier pruning. GraphQL [27] and QuickSI [46] introduced sophisticated structural filtering (e.g., neighborhood

signatures) and infrequent-edge-first matching orders, respectively. CECI [7] proposes a compact embedding cluster index and utilizes set intersections for fast candidate resolution. CFL-Match [8] decomposes the query into core, forest, and leaf structures to delay the Cartesian products of leaves. DPiso [24] utilizes dynamic programming based on Directed Acyclic Graphs (DAG) to generate robust candidate spaces. Algorithm IVE [31] separates isolated query vertices from the matching order to accelerate the enumeration.

Close to this work are VEQ [35], TurboISO [25] and B $\otimes$ X [40]. Algorithm VEQ uses dynamic equivalences to support backward-looking pruning. Algorithm TurboISO merges similar query vertices into Neighborhood Equivalence Classes to construct a compacted search region. Algorithm B $\otimes$ X [40] groups structurally similar vertices into search boxes and processes them in batches.

This work departs from prior work as follows. (1) We propose the vertex-set simulation to group vertices, which is not studied in prior work. (2) We construct a contraction graph from the vertex groups, while existing algorithms directly develop strategies on the original graph to reduce searching space during backtracking, e.g., [13, 25, 27, 35, 40]. (3) Existing matching algorithms can be extended to deal with contraction graphs without increasing complexity.

Graph simulation. Multiple simulation have been proposed to relax the isomorphic semantics for subgraph matching. Graph simulation [17, 28] only requires to preserve adjacent edges for vertices on undirected graphs. Dual simulation [41] is extended to directed graphs, and preserves both incoming and outgoing edges of vertices. Graded simulation [44] further requires to preserve the numbers of incoming and outgoing edges. Strong simulation [41] enforces a locality constraint via a bounding ball. Strict simulation [21] and tight simulation [29] further refine structural size constraints. Parametric simulation links entities across relational databases and graphs by aggregating closeness of descendants of vertices [18].

This work differs from prior work as follows: (1) Vertex-set simulation incorporates triangle checking in the definition, which improve the filtering quality and is not studied in the prior work. (2) A query vertex is mapped to a set of vertices in the graphs, while other models map a query vertex to only one vertex in graphs. (3) The maximal vertex-set simulation is not unique, while the simulation models have a unique maximum relation. We also prove that there exists a unique minimum maximal vertex-set simulation, which is not studied in the prior work.

7 CONCLUSION

This paper proposed Algorithm VSMatch for subgraph enumeration. To accelerate the computation, we proposed the vertex-set simulation, which groups locally indistinguishable vertices, and develop an algorithm to compute the vertex-set simulation. Based on this, we developed Algorithm VSMatch, which compresses the original graphs based on vertex-set simulation, and performs enumeration on the compressed search space induced by compressed graphs. Experiments on real and synthetic graphs show that VSMatch outperforms state-of-the-art algorithms and achieves up to 23 orders of magnitude improvement in EPS metric.

One topic for future work is to develop incremental matching algorithms in response to updates to graphs. Another topic is to extend the study to multi-query optimization.

REFERENCES

- [1] 2026. Code, datasets and full version. <https://github.com/SU-ZQ/VSMatch>.
- [2] Christopher R. Aberger, Susan Tu, Kunle Olukotun, and Christopher Ré. 2016. EmptyHeaded: A Relational Engine for Graph Processing. In *SIGMOD*. 431–446.
- [3] Wei Ai, CanHao Xie, Tao Meng, Jiayi Du, and Keqin Li. 2024. A D-Truss-Equivalence Based Index for Community Search Over Large Directed Graphs. *TKDE* 36, 11 (2024), 5482–5494.
- [4] Waqas Ali, Muhammad Saleem, Bin Yao, Aidan Hogan, and Axel-Cyrille Ngonga Ngomo. 2022. A survey of RDF stores & SPARQL engines for querying knowledge graphs. *VLDBJ* 31, 3 (2022), 1–26.
- [5] Khaled Ammar, Frank McSherry, Semih Salihoglu, and Manas Joglekar. 2018. Distributed Evaluation of Subgraph Queries Using Worst-case Optimal and Low-Memory Dataflows. *PVLDB* 11, 6 (2018), 691–704.
- [6] Junya Arai, Yasuhiro Fujiwara, and Makoto Onizuka. 2023. GuP: Fast Subgraph Matching by Guard-based Pruning. *Proc. ACM Manag. Data* 1, 2 (2023), 167:1–167:26.
- [7] Bibek Bhattarai, Hang Liu, and H. Howie Huang. 2019. CECI: Compact Embedding Cluster Index for Scalable Subgraph Matching. In *SIGMOD*, Peter Boncz, Stefan Manegold, Anastasia Ailamaki, Amol Deshpande, and Tim Kraska (Eds.). 1447–1462.
- [8] Fei Bi, Lijun Chang, Xuemin Lin, Lu Qin, and Wenjie Zhang. 2016. Efficient Subgraph Matching by Postponing Cartesian Products. In *SIGMOD*, Fatma Özcan, Georgia Koutrika, and Sam Madden (Eds.). 1199–1214.
- [9] Vincenzo Bonnici, Rosalba Giugno, Alfredo Pulvirenti, Dennis E. Shasha, and Alfredo Ferro. 2013. A subgraph isomorphism algorithm and its application to biochemical data. *BMC Bioinform.* 14, S-7 (2013), S13.
- [10] Vincenzo Bonnici, Rosalba Giugno, Alfredo Pulvirenti, Dennis E. Shasha, and Alfredo Ferro. 2013. A subgraph isomorphism algorithm and its application to biochemical data. *BMC Bioinform.* 14, S-7 (2013), S13.
- [11] Vincenzo Carletti, Pasquale Foggia, Alessia Saggese, and Mario Vento. 2017. Introducing VF3: A New Algorithm for Subgraph Isomorphism. In *GRRP*. 128–139.
- [12] Yunyoung Choi, Kunsoo Park, and Hyunjoon Kim. 2023. BICE: Exploring Compact Search Space by Using Bipartite Matching and Cell-Wide Verification. *PVLDB* 16, 9 (2023), 2186–2198.
- [13] Luigi P. Cordella, Pasquale Foggia, Carlo Sansone, and Mario Vento. 2004. A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs. *TPAMI* 26, 10 (2004), 1367–1372.
- [14] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2022. *Introduction to algorithms*. MIT press.
- [15] Kayhan Erciyes. 2023. Graph-Theoretical Analysis of Biological Networks: A Survey. *Comput.* 11, 10 (2023), 188.
- [16] Wenfei Fan. 2012. Graph pattern matching revised for social network analysis. In *ICDE*, Alin Deutsch (Ed.). 8–21.
- [17] Wenfei Fan, Jianzhong Li, Shuai Ma, Nan Tang, Yinghui Wu, and Yunpeng Wu. 2010. Graph Pattern Matching: From Intractable to Polynomial Time. *PVLDB* 3, 1 (2010), 264–275.
- [18] Wenfei Fan, Ping Lu, Kehan Pang, Ruochun Jin, and Wenyuan Yu. 2024. Linking Entities across Relations and Graphs. *TODS* 49, 1 (2024), 2:1–2:50.
- [19] Wenfei Fan, Yixuan Luo, Ping Lu, and Xiaoke Zhu. 2026. Enumerating Graph Pattern Matches with ML Oracles. *Proc. ACM Manag. Data* 4, 1, Article 34 (April 2026), 28 pages. <https://doi.org/10.1145/3786648>
- [20] Binbin Fang, Hanzhu Chen, Weiwei Wang, and Yansong Wang. 2024. GraphFA: Graph Enhanced Fraud Detectors with Camouflage Detection for Financial Anti-Fraud. In *ICSP*. 323–327.
- [21] Arash Fard, M. Usman Nisar, Lakshminish Ramaswamy, John A. Miller, and Matthew Saltz. 2013. A distributed vertex-centric approach for pattern matching in massive graphs. In *BigData*, Xiaohua Hu, Tsau Young Lin, Vijay V. Raghavan, Benjamin W. Wah, Ricardo Baeza-Yates, Geoffrey C. Fox, Cyrus Shahabi, Matthew Smith, Qiang Yang, Rayid Ghani, Wei Fan, Ronny Lempel, and Raghunath Nambiar (Eds.). 403–411.
- [22] M. R. Garey and David S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- [23] Qiuyu Guo, Jianye Yang, Wenjie Zhang, Hanchen Wang, Ying Zhang, and Xuemin Lin. 2025. Efficient and Accurate Subgraph Counting: A Bottom-up Flow-learning based Approach. *Proc. VLDB Endow.* 18, 8 (2025), 2695–2708.
- [24] Myoungji Han, Hyunjoon Kim, Geonmo Gu, Kunsoo Park, and Wook-Shin Han. 2019. Efficient Subgraph Matching: Harmonizing Dynamic Programming, Adaptive Matching Order, and Failing Set Together. In *SIGMOD*. 1429–1446.
- [25] Wook-Shin Han, Jinsoo Lee, and Jeong-Hoon Lee. 2013. Turbo_{iso}: towards ultrafast and robust subgraph isomorphism search in large graph databases. In *SIGMOD*, Kenneth A. Ross, Divesh Srivastava, and Dimitris Papadias (Eds.). 337–348.
- [26] W. K. Hastings. 1970. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika* 57, 1 (1970), 97–109.
- [27] Huahai He and Ambuj K. Singh. 2008. Graphs-at-a-time: query language and access methods for graph databases. In *SIGMOD*, Jason Tsong-Li Wang (Ed.). 405–418.
- [28] M.R. Henzinger, T.A. Henzinger, and P.W. Kopke. 1995. Computing simulations on finite and infinite graphs. In *FOCS*. 453–462.
- [29] Arash Jalal Zadeh Fard, M Nisar, John Miller, and Lakshminish Ramaswamy. 2014. Distributed and Scalable Graph Pattern Matching: Models and Algorithms. *International Journal of Big Data* 1 (01 2014), 1–14.
- [30] Haolin Jiang, Santosh Pandey, and Hang Liu. 2025. A Comprehensive Survey of Subgraph Matching: [Experiments & Analysis]. *Proc. ACM Manag. Data* 3, 6 (2025), 1–30.
- [31] Zite Jiang, Shuai Zhang, Xingzhong Hou, Mengting Yuan, and Haihang You. 2024. IVE: Accelerating Enumeration-Based Subgraph Matching via Exploring Isolated Vertices. In *ICDE*. 4208–4221.
- [32] Tatiana Jin, Boyang Li, Yichao Li, Qihui Zhou, Qianli Ma, Yunjian Zhao, Hongzhi Chen, and James Cheng. 2023. Circinus: Fast Redundancy-Reduced Subgraph Matching. *Proc. ACM Manag. Data* 1, 1 (2023), 12:1–12:26.
- [33] Alpar Jüttner and Péter Madarasi. 2018. VF2++ - An improved subgraph isomorphism algorithm. *Discret. Appl. Math.* 242 (2018), 69–81.
- [34] Hasara Kalumin and Amol Deshpande. 2025. Optimizing Queries with Many-to-Many Joins. In *ICDE*. 3668–3681.
- [35] Hyunjoon Kim, Yunyoung Choi, Kunsoo Park, Xuemin Lin, Seok-Hee Hong, and Wook-Shin Han. 2021. Versatile Equivalences: Speeding up Subgraph Query Processing and Subgraph Matching. In *SIGMOD*, Guoliang Li, Zhanhuai Li, Stratos Idreos, and Divesh Srivastava (Eds.). 925–937.
- [36] Jinha Kim, Hyungyu Shin, Wook-Shin Han, Sungpack Hong, and Hassan Chafi. 2015. Taming Subgraph Isomorphism for RDF Query Processing. *PVLDB* 8, 11 (2015), 1238–1249.
- [37] Jinsoo Lee, Wook-Shin Han, Romans Kasperovics, and Jeong-Hoon Lee. 2012. An In-depth Comparison of Subgraph Isomorphism Algorithms in Graph Databases. *PVLDB* 6, 2 (2012), 133–144.
- [38] Jinsoo Lee, Wook-Shin Han, Romans Kasperovics, and Jeong-Hoon Lee. 2012. An In-depth Comparison of Subgraph Isomorphism Algorithms in Graph Databases. *PVLDB* 6, 2 (2012), 133–144.
- [39] Mingdao Li, Peng Peng, Zheyuan Hu, Lei Zou, and Zheng Qin. 2024. Variable-Length Path Query Evaluation Based on Worst-Case Optimal Joins. In *ICDE*. 3311–3323.
- [40] Yujie Lu, Zhijie Zhang, and Weiguo Zheng. 2025. B(S)X: Subgraph Matching with Batch Backtracking Search. *Proc. ACM Manag. Data* 3, 1 (2025), 15:1–15:27.
- [41] Shuai Ma, Yang Cao, Wenfei Fan, Jinpeng Huai, and Tianyu Wo. 2011. Capturing Topology in Graph Pattern Matching. *PVLDB* 5, 4 (2011), 310–321.
- [42] Shuai Ma, Yang Cao, Wenfei Fan, Jinpeng Huai, and Tianyu Wo. 2014. Strong simulation: Capturing topology in graph pattern matching. *TODS* 39, 1 (2014), 4:1–4:46.
- [43] Amine Mhedhbi and Semih Salihoglu. 2019. Optimizing Subgraph Queries by Combining Binary and Worst-Case Optimal Joins. *PVLDB* 12, 11 (2019), 1692–1704.
- [44] Martin Otto. 2020. Graded modal logic and counting bisimulation. *CoRR* arXiv:1910.00039 (2020). <https://doi.org/10.48550/arXiv.1910.00039>
- [45] Himchan Park and Min-Soo Kim. 2018. EvoGraph: An Effective and Efficient Graph Upscaling Method for Preserving Graph Properties. In *KDD*, Yike Guo and Faisal Farooq (Eds.). 2051–2059.
- [46] Haichuan Shang, Ying Zhang, Xuemin Lin, and Jeffrey Xu Yu. 2008. Taming verification hardness: an efficient algorithm for testing subgraph isomorphism. *PVLDB* 1, 1 (2008), 364–375.
- [47] Wonseok Shin, Siwoo Song, Kunsoo Park, and Wook-Shin Han. 2024. Cardinality Estimation of Subgraph Matching: A Filtering-Sampling Approach. *Proc. VLDB Endow.* 17, 7 (2024), 1697–1709.
- [48] Junshuai Song, Xiaoru Qu, Zehong Hu, Zhao Li, Jun Gao, and Ji Zhang. 2021. A subgraph-based knowledge reasoning method for collective fraud detection in E-commerce. *Neurocomputing* 461 (2021), 587–597.
- [49] Shixuan Sun and Qiong Luo. 2020. In-Memory Subgraph Matching: An In-depth Study. In *SIGMOD*, David Maier, Rachel Pottinger, AnHai Doan, Wang-Chiew Tan, Abdussalam Alawini, and Hung Q. Ngo (Eds.). 1083–1098.
- [50] Julian R. Ullmann. 1976. An Algorithm for Subgraph Isomorphism. *JACM* 23, 1 (1976), 31–42.
- [51] Rongjian Yang, Zhijie Zhang, Weiguo Zheng, and Jeffrey Xu Yu. 2023. Fast Continuous Subgraph Matching over Streaming Graphs via Backtracking Reduction. *Proc. ACM Manag. Data* 1, 1 (2023), 15:1–15:26.
- [52] Qi Zhang, Rong-Hua Li, Hongchao Qin, Guoren Wang, Zhiwei Zhang, and Ye Yuan. 2023. Stable Subgraph Isomorphism Search in Temporal Networks. *TKDE* 35, 6 (2023), 6405–6420.
- [53] Zhijie Zhang, Yujie Lu, Weiguo Zheng, and Xuemin Lin. 2024. A Comprehensive Survey and Experimental Study of Subgraph Matching: Trends, Unbiasedness, and Interaction. *Proc. ACM Manag. Data* 2, 1 (2024), 60:1–60:29.

APPENDIX

A. Proof of Theorem 1

We prove it by contradiction. Assume that $\mathcal{R}_1$ and $\mathcal{R}_2$ are two distinct minimum maximal vertex-set simulation relations. We define a smaller vertex-set simulation relation $\mathcal{R}$ using $\mathcal{R}_1$ and $\mathcal{R}_2$, and show that $\mathcal{R}$ is a maximal vertex-set simulation, a contradiction.

Given $\mathcal{R}_1$ and $\mathcal{R}_2$, we construct $\mathcal{R}$ as follows. Let $\mathcal{R}_{\text{union}} = \mathcal{R}_1 \cup \mathcal{R}_2$. For any query vertex $u \in V_Q$, we construct the set $\mathcal{V}_u = \{V \mid (u, V) \in \mathcal{R}_{\text{union}}\}$. We construct $\mathcal{R}$ by merging sets in $\mathcal{V}_u$ that having the same neighbors with respect to Q . More formally, for any two distinct pairs (u, V_A) and (u, V_B) in $\mathcal{R}_{\text{union}}$, if V_A and V_B share the same neighbors with respect to Q , i.e., for any neighbor u_n of u in Q and any pair (u_n, V_n) , every vertex in V_A connects to each vertex in V_n if and only if every vertex in V_B does. That is,

$$\forall u_n \in N_Q(u), \forall V_n \in \mathcal{V}_{u_n}, (V_A \times V_n \subseteq E_G \iff V_B \times V_n \subseteq E_G)$$

We replace (u, V_A) and (u, V_B) with $(u, V_A \cup V_B)$ to $\mathcal{R}_{\text{union}}$. When no two pairs can be merged, we denote the resulted set as $\mathcal{R}$.

We deduce the contradiction by showing that (a) $\mathcal{R}$ is a vertex-set simulation; (b) $\mathcal{R}$ is maximal; and (c) $|\mathcal{R}| < |\mathcal{R}_1|$ or $|\mathcal{R}| < |\mathcal{R}_2|$.

(I) We first show that $\mathcal{R}$ is a vertex-set simulation. We prove that $\mathcal{R}$ satisfies the three conditions for the definition of vertex-set simulation. (i) For each pair $(u, V) \in \mathcal{R}$, since V is the union of vertices in the pair (u, V) from $\mathcal{R}_1$ or $\mathcal{R}_2$ and both $\mathcal{R}_1$ or $\mathcal{R}_2$ are vertex-set simulation. all vertices in V carry the same label as u . Moreover, $L_Q(u) = L_G(v)$ for each $v \in V$. (ii) For each query edge $(u, u') \in E_Q$, we show the following two properties: (A) given any two pairs (u, V_1) and (u', V_2) from $\mathcal{R}$, if there exists an edge between v_1 and v_2 with $v_1 \in V_1$ and $v_2 \in V_2$, then every vertex in V_1 has an edge to each vertex in V_2 ; and (B) there exist two elements (u, V'_1) and (u', V'_2) in $\mathcal{R}$ such that all vertices in V'_1 have an edge to each vertex in V'_2 . We start with (A). The set V_1 consists of all vertices having the same neighbors w.r.t. Q , then if there exists an edge between v_1 and v_2 , all vertices in V_1 have an edge to v_2 . Similarly, since V_2 consists of all vertices having the same neighbors w.r.t. Q , all vertices in V_1 have an edge to each vertex in V_2 . For (B), since both $\mathcal{R}_1$ is a vertex-set simulation, there exists (u, V_1^1) and (u', V_2^1) from $\mathcal{R}_1$ such that all vertices in V_1^1 have an edge to each vertex in V_2^1 . Because $\mathcal{R}$ is constructed by merging vertex sets with the same neighbors w.r.t. Q , there exists (u, V'_1) and (u', V'_2) from $\mathcal{R}$ such that $V_1^1 \subseteq V'_1$ and $V_2^1 \subseteq V'_2$. Due to property (A), that all vertices in V'_1 have an edge to each vertex in V'_2 , i.e., property (B) holds. The triangle relations(C3) can be proved similarly. We omit the details here.

(2) We show that $\mathcal{R}$ is maximal. Let $C_{\mathcal{R}}(u) = \bigcup_{(u, V) \in \mathcal{R}} V$ denote the set of all vertices that can be matched by u in vertex-set simulation. It is to show there does not exist another vertex-set simulation relation $\mathcal{R}'$, under which query vertex u match more vertices in G than than under $\mathcal{R}$, i.e., $C_{\mathcal{R}}(u) \subset C_{\mathcal{R}'}(u)$ for some $u \in V_Q$. Assume by contradiction that such $\mathcal{R}'$ exists. Because each $(u, V) \in \mathcal{R}$ is constructed by merging vertex sets in $\mathcal{R}_1 \cup \mathcal{R}_2$, $C_{\mathcal{R}_1}(u) \cup C_{\mathcal{R}_2}(u) \subseteq C_{\mathcal{R}}(u)$ holds. Therefore, $C_{\mathcal{R}_1}(u) \subset C_{\mathcal{R}'}(u)$ and $C_{\mathcal{R}_2}(u) \subset C_{\mathcal{R}'}(u)$, i.e., neither $\mathcal{R}_1$ nor $\mathcal{R}_2$ is maximal, a contradiction.

(3) We prove that $|\mathcal{R}| < |\mathcal{R}_1|$ or $|\mathcal{R}| < |\mathcal{R}_2|$. We prove this property in the following two steps: (I) $C_{\mathcal{R}_1}(u) = C_{\mathcal{R}_2}(u)$, i.e., the sets of vertices that can be matched to u are the same; and (II)

(I) We first show that $C_{\mathcal{R}_1}(u) = C_{\mathcal{R}_2}(u)$. Assume by contradiction that there exists a pair $(u, V_1) \in \mathcal{R}_1$ and a vertex $v_o \in V_1$ such that no $(u, V_2) \in \mathcal{R}_2$ satisfies $v_o \in V_2$, i.e., $C_{\mathcal{R}_1}(u) \setminus C_{\mathcal{R}_2}(u) \neq \emptyset$. We can show that $\mathcal{R}_2$ is not maximal.

We construct a new relation $\mathcal{R}'$:

$$\mathcal{R}' = \{(u, \{v\}) \mid u \in V_Q \text{ and } v \in C_{\mathcal{R}_1}(u) \cup C_{\mathcal{R}_2}(u)\}$$

We now verify that $\mathcal{R}'$ is a vertex-set simulation relation. Observe that every vertex set $\{v\}$ consists of only one vertex, and v is from either $C_{\mathcal{R}_1}(u)$ or $C_{\mathcal{R}_2}(u)$. It is easy verify that $\mathcal{R}'$ satisfies the three conditions for vertex-set simulation relation. Because $v_o \notin C_{\mathcal{R}_2}(u)$, $\mathcal{R}_2$ is not maximal, a contradiction. Therefore, $C_{\mathcal{R}_1}(u) = C_{\mathcal{R}_2}(u)$.

(II) We next show that $|\mathcal{R}| < |\mathcal{R}_1|$ or $|\mathcal{R}| < |\mathcal{R}_2|$. Since $\mathcal{R}_1$ and $\mathcal{R}_2$ are distinct maximal vertex-set simulations, assume w.l.o.g. that there exist a query vertex u and a vertex set V such that $(u, V_o) \in \mathcal{R}_1$ but $(u, V_o) \notin \mathcal{R}_2$. From property (I), $C_{\mathcal{R}_1}(u) = C_{\mathcal{R}_2}(u)$. Consider the following two cases: (i) V is the union of multiple vertex sets in $\mathcal{R}_1$, i.e., there exist $V_1, \dots, V_k$ in $\mathcal{R}_2$ such that $V_o = V_1 \cup \dots \cup V_k$; or (ii) there exists a vertex set V_o in $\mathcal{R}_2$ such that $V_c \cap V_o \neq \emptyset$ and $V_o \not\subseteq V_c$. For case (i) Because $\mathcal{R}_1$ is a vertex-set simulations, all vertices in V_c have the same neighbors w.r.t. Q . Then all $V_1, \dots, V_k$ are merged in $\mathcal{R}$. Therefore, $|\mathcal{R}| < |\mathcal{R}_2|$. For case (ii), because $C_{\mathcal{R}_1}(u) = C_{\mathcal{R}_2}(u)$, $V_c \cap V_o \neq \emptyset$ and $V_o \not\subseteq V_c$, there exist two vertices $v_1, v_2 \in C_{\mathcal{R}_1}(u) = C_{\mathcal{R}_2}(u)$ and a pair $(u, V_e) \in \mathcal{R}_1$ such that $v \in V_c \cap V_o$, $v' \notin V_c$ and $v' \in V_o \cap V_e$. Because $v \in V_c \cap V_o$, $\mathcal{R}_1$ and $\mathcal{R}_2$ are vertex-set simulations, all vertices in V and V_1 have the same neighbors w.r.t. Q . Then V_c and V_o are merged in $\mathcal{R}$. Similarly, V_o and V_e can be merged in $\mathcal{R}$; that is V_c and V_e are merged in $\mathcal{R}$. Therefore, $|\mathcal{R}| < |\mathcal{R}_1|$.